Основы анализа данных

Получение данных из Excel и API

Файлы, запросы и сохранённые ответы

Формат и способ получения

Формат описывает содержимое
API задаёт правила запроса и ответа

Представление	Что внутри
CSV	Одна плоская таблица
Excel	Книга с листами
JSON	Вложенные словари и списки
API	Правила обращения программы к сервису

Excel-книга и выбор листа

sheet_name выбирает лист

usecols оставляет нужные столбцы

```
book = pd.read_excel(
    LOCAL / "hmda_exchange.xlsx", sheet_name=None
)
print({name: table.shape for name, table in book.items()})
```

Ноутбук: B03

Как устроен JSON

Фрагмент сохранённого ответа GLEIF: код и имя


```
{"data": [
  {"id": "549300P1347BFCBHJV60",
   "attributes": {"entity": {
     "legalName": {"name": "MASS. INSTITUTE OF TECH. FEDERAL CREDIT UNION"}
   }}}
]}
```

Чтение сохранённого ответа

json.load читает открытый файл

json.loads разбирает содержимое строки

```
with (LOCAL / "gleif_page_01.json").open(encoding="utf-8") as file:
    page1 = json.load(file)
print(type(page1), list(page1))
```

Ноутбук: B05

Одна запись внутри ответа

page1["data"] — список организаций

[0] выбирает первую запись списка

```
record = page1["data"][0]
record["id"]
record["attributes"]["entity"]["legalName"]["name"]
```

Ноутбук: B06

Список записей превращается в таблицу

record_path указывает список для строк DataFrame

```
orgs = pd.json_normalize(filers, record_path="institutions")
orgs.head(3)
```

Ноутбук: B07

Вложенные поля становятся столбцами

Точка в названии показывает путь к значению

```
flat1 = pd.json_normalize(page1["data"], sep=".")
cols = ["id", "attributes.entity.legalName.name",
        "attributes.entity.legalAddress.country"]
flat1[cols]
```

Ноутбук: B08

Список других имён

Одна организация может иметь несколько имён

После разворота одна строка описывает одно имя

```
aliases = pd.json_normalize(
    page1["data"],
    record_path=["attributes", "entity", "otherNames"],
    meta=["id"]
)
```

Ноутбук: B09

Проверка полученной таблицы

Нужные столбцы, известные и уникальные LEI

Отдельно проверяем полноту результата

```
assert required.issubset(flat1.columns)
assert flat1["id"].notna().all()
assert flat1["id"].is_unique
```

Ноутбук: B10

Запрос GET и его параметры

Адрес задаёт ресурс, params — условия запроса

```
params = {"filter[lei]": ",".join(selected), "page[size]": 2}
prepared = requests.Request("GET", endpoint, params=params).prepare()
print(prepared.url)
```

Ноутбук: B11

Выполнение запроса requests.get

timeout ограничивает ожидание ответа

RUN_LIVE=False оставляет занятие без сети

```
RUN_LIVE = False
if RUN_LIVE:
    live_response = requests.get(
        endpoint, params=params, timeout=(5, 30)
    )
```

Ноутбук: B12

Что содержит HTTP-ответ

Статус и заголовки описывают ответ

Тело содержит данные или сообщение сервиса

```
print(response.status_code)
print(response.headers.get("Content-Type"))
response.raise_for_status()
payload = response.json()
```

Ноутбук: B13

Ошибка сервиса и следующий шаг

При ошибке сначала читаем статус и причину

Статус или сбой	Что проверить
200	Формат, структуру и полноту тела
400 / 404	Параметры и адрес
429	Ограничение частоты, Retry-After
5xx / ошибка соединения	Причину сбоя и допустимость повтора

Ноутбук: B14

Проверки ответа по порядку

HTTP → формат → JSON → нужная структура

```
payload = decode_json_response(response)
print(type(payload), list(payload))
```

Ноутбук: B15

Результат на нескольких страницах

Первая страница содержит 2 записи, вторая — 1

links.next указывает продолжение

```
page1["links"].get("next")
page2["links"].get("next")
all_records = page1["data"] + page2["data"]
```

Ноутбук: B16

Обход страниц в одной функции

Загрузить страницу, добавить записи, перейти по next

```
collected = collect_pages(manifest["first_url"], cached_loader)
assert {r["id"] for r in collected} == set(selected)
```

Ноутбук: B17

Сохранённые ответы и manifest

Исходные байты и сведения о получении хранятся вместе

```
body = (LOCAL / info["file"]).read_bytes()
assert hashlib.sha256(body).hexdigest() == info["sha256"]
```

Ноутбук: B18

Добавление сведений GLEIF

Три LEI покрывают часть ипотечных записей

left JOIN сохраняет весь исходный набор

```
enriched = decisions.merge(
    lookup, on="lei", how="left",
    validate="many_to_one", indicator=True
)
enriched["_merge"].value_counts()
```

Ноутбук: B19

Год записи и время получения справочника

HMDA: отчётный год 2023

GLEIF: сохранённый ответ от 10 октября 2026 года

```
pd.to_datetime(acquisition["downloaded_at_utc"], utc=True)
pd.to_datetime(acquisition["golden_copy_publish_date"], utc=True)
```

Ноутбук: B20

Повторный запуск и новая загрузка

Сохранённый снимок нужен для воспроизведения
Новая загрузка создаёт отдельную версию

Действие	Какие файлы используем
Повтор анализа	Те же ответы и manifest
Новое получение	Новая папка ответов и новый manifest
Сбой получения	Сообщение с причиной; прежние файлы сохранены

Вопрос учебного проекта

Вопрос, данные и способ расчёта должны подходить друг другу

proposal["question"]
proposal["population"]
proposal["metric"]

Ноутбук: B22

Практика получения данных

Прочитать, проверить и сохранить справочник